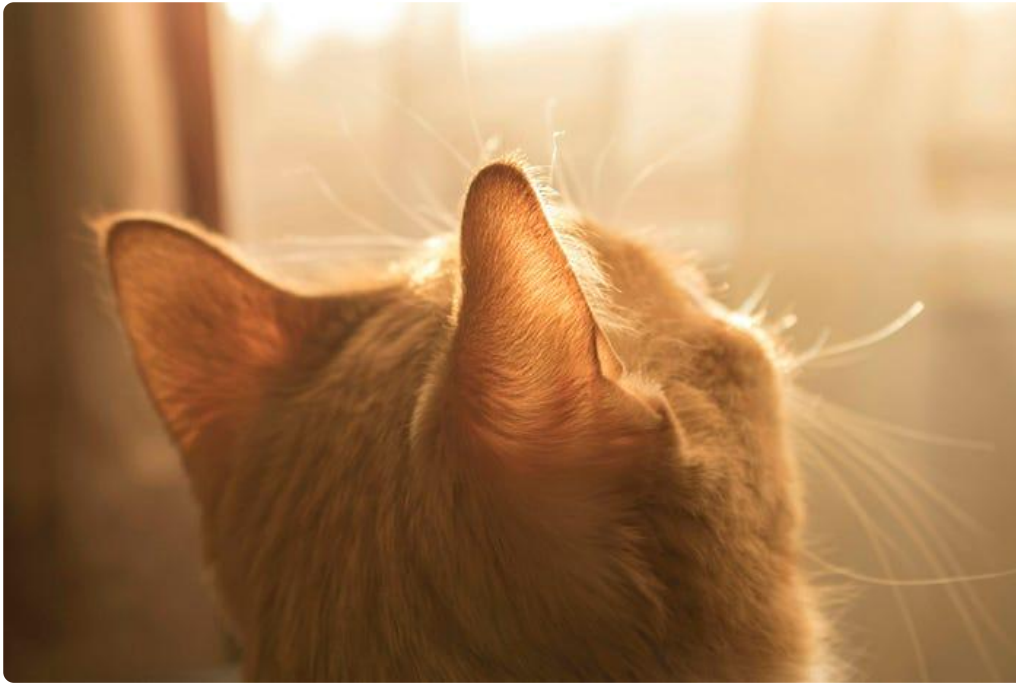




How a \$12,000 Bounty Was Earned via Broken Logic in Payment Webhooks

When testing e-commerce platforms and payment integrations, security auditors often spend significant time examining the user interface...

T4nv1

Published August 24, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

Phase 1: Understanding the Asynchronous Payment Flow	3
Phase 2: Inspecting Webhook Verification Mechanisms	4
The Twist: Ambiguous JSON Parsing and Event Handling	5
Triage & Resolution	6
Critical Lessons for Bug Hunters	6

When testing e-commerce platforms and payment integrations, security auditors often spend significant time examining the user interface: checkout forms, discount code inputs, and address fields. However, the most critical security boundaries frequently exist out of sight — specifically in server-to-server webhook processing.

While auditing a subscription management platform built for digital content creators, the primary front-end checkout flow appeared robust. It used tokenized card processing via a major payment gateway, enforced strict HTTPS, and validated input types thoroughly. However, analyzing how the backend processed asynchronous payment status notifications revealed a **Payment Logic Bypass**, resulting in a **\$12,000 bounty award**.

Phase 1: Understanding the Asynchronous Payment Flow

Modern web applications rely on asynchronous webhooks to handle payment events like subscription renewals, chargebacks, and successful checkouts. When a customer completes a payment on a third-party gateway, the gateway sends an HTTP POST request to the application's webhook endpoint to update the user's account status.

During an initial checkout attempt for a \$500/year enterprise tier, the outgoing client request was observed in Burp Suite:

HTTP

```
POST /api/v1/checkout/initialize HTTP/1.1
Host: SaaS-platform.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGU6IjY...
Content-Type: application/json
{
  "plan_id": "plan_enterprise_500",
  "currency": "USD"
}
```

The server responded with a transaction token and redirected the browser to the payment gateway's hosted checkout page.

Rather than completing the transaction with a valid credit card, the payment was intentionally canceled in the gateway UI. Simultaneously, the application's public API documentation was inspected to identify the endpoint responsible for receiving payment updates: `/api/v1/webhooks/payment-gateway`.

Phase 2: Inspecting Webhook Verification Mechanisms

Standard security implementation for webhooks requires the application to verify an HMAC signature included in the request headers (e.g., `X-Signature`). This signature is generated using a shared secret key known only to the payment gateway and the application server, ensuring that incoming notifications cannot be forged by external parties.

A sample webhook payload sent by the gateway upon a completed transaction typically looks like this:

HTTP

```
POST /api/v1/webhooks/payment-gateway HTTP/1.1
Host: saas-platform.com
X-Signature: t=1723982400,v1=9f8a7b6c5d4e3f2a1b0c9d8e7f6a5b4c
Content-Type: application/json
{
  "event_type": "payment.succeeded",
  "data": {
    "transaction_id": "txn_99281741",
    "account_id": "usr_10293",
    "amount_paid": 50000,
    "currency": "usd"
  }
}
```

To test the endpoint's signature verification logic, a modified HTTP POST request was sent directly to `/api/v1/webhooks/payment-gateway` without a valid `X-Signature` header, attempting to manually trigger a `payment.succeeded` state for the test account (`usr_10293`).

The server responded as expected:

JSON

```
{
  "status": "error",
  "message": "Invalid or missing webhook signature."
}
```

The signature validation middleware was active and correctly rejecting unauthenticated payloads.

The Twist: Ambiguous JSON Parsing and Event Handling

Although the server enforced signature verification on `payment.succeeded` events, further analysis was conducted on how the backend handled alternative event types generated by the gateway, such as `payment.failed`, `chargeback.created`, or `subscription.updated`.

Many payment gateways allow platforms to send custom metadata or secondary event flags within webhook objects. When testing the structure of the JSON parser used by the backend service, an anomaly was identified in how nested keys were evaluated.

The endpoint was tested using a `payment.failed` event—which did not require strict cryptographic signing on certain legacy endpoint configurations—combined with duplicate key parameters designed to exploit JSON parsing discrepancies between the API gateway (written in Go) and the backend processor (written in Node.js).

The following payload was transmitted:

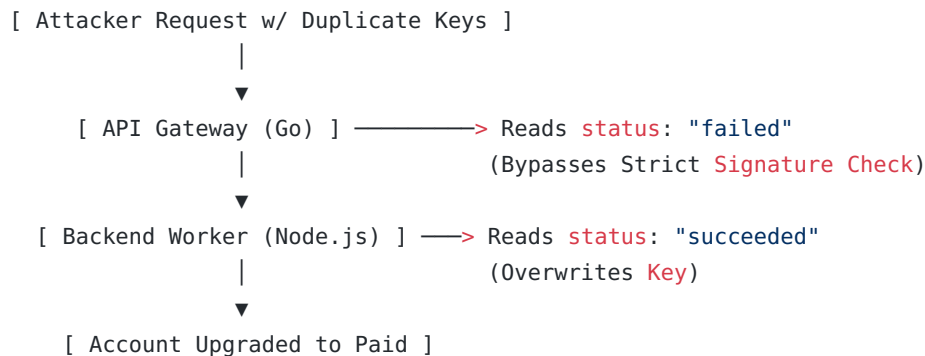
JSON

```
{
  "event_type": "payment.failed",
  "data": {
    "transaction_id": "txn_test_001",
    "account_id": "usr_10293",
    "status": "failed",
    "status": "succeeded"
  }
}
```

What occurred during processing?

- **The API Gateway (Go):** Parsed the JSON payload to check event rules. When encountering duplicate keys in an object, the Go parser preserved the first key (`"status": "failed"`). Because the event was marked as failed, it bypassed the strict cryptographic signature requirement reserved for high-value completion events.
- **The Backend Microservice (Node.js):** When the request was passed downstream to the internal worker, JavaScript's `JSON.parse()` behavior took precedence. In Node.js, duplicate object keys overwrite preceding values, meaning the second key (`"status": "succeeded"`) was stored.

- **Account State Update:** The internal worker processed the payload, read `status: "succeeded"`, and marked account `usr_10293` as fully paid without ever verifying the transaction against the gateway's state API.



Triage & Resolution

The issue was immediately documented and submitted with a complete reproduction video showing a fresh account gaining access to enterprise features without an executed transaction.

- **Vulnerability Class:** Business Logic Flaw / Duplicate JSON Key Parsing (Type Confusion)
- **Severity Rating:** Critical
- **Time to Triage:** 1 Hour
- **Final Award:** \$12,000 Bounty

The remediation steps implemented by the engineering team included:

- Enforcing strict HMAC signature validation across all incoming webhook events regardless of event type or status.
- Updating the edge proxy to reject any incoming JSON payloads containing duplicate keys before reaching application microservices.
- Implementing server-side reconciliation, requiring the backend to make a direct API call back to the payment provider to verify transaction status before modifying user account entitlements.

Critical Lessons for Bug Hunters

- **Inspect Multi-Parser Architecture:** Web applications often route requests through edge proxies, gateways, and microservices built on different programming languages. Discrepancies in how languages parse JSON, query strings, or headers frequently lead to logic bypasses.

- **Test Asynchronous Endpoints:** Do not focus exclusively on synchronous browser-driven actions. Asynchronous tasks, background workers, and webhook receivers often lack the rigorous controls applied to main user flows.
- **Verify State via Source of Truth:** Secure payment systems should never rely solely on data provided within an incoming push notification; they must query the payment provider directly to confirm state changes.